

(17)

K-nearest neighbor

Majority voting of neighbors.

 $N_k(x) \Rightarrow$ indices of k -NN of x in the dataset.

Regression: $\hat{y}(x) = \frac{1}{k} \sum_{i \in N_k(x)} y_i$

Classification: $\hat{y}(x) = \arg \max_c \frac{1}{k} \sum_{i \in N_k(x)} \mathbb{I}(y_i = c)$

indicator function
+1 if $y_i = c$
0 if $y_i \neq c$

Weighted NNfor a query point x ,

$$\hat{y}(x) = \frac{\sum_i w_i y_i}{\sum_i w_i}, \text{ where } w_i = \exp\left(-\frac{\|x - x_i\|^2}{2\sigma^2}\right)$$

When $\sigma \rightarrow \infty$, $\frac{\|x - x_i\|^2}{2\sigma^2} \rightarrow 0$

$$\Rightarrow w_i \rightarrow \exp(0) = 1$$

So all weights become equal.

Hence, $\hat{y}(x) = \frac{\sum_i y_i \cdot 1}{\sum_i 1} = \frac{1}{n} \sum_i y_i = \text{Mean of the dataset}$

When $\sigma \rightarrow 0$ If $x \neq x_i$, then $\|x - x_i\|^2 > 0$
As $\sigma \rightarrow 0$, denominator $2\sigma^2 \rightarrow 0$
So, $\|x - x_i\|^2 / 2\sigma^2 \rightarrow \infty$

$$\Rightarrow w_i \rightarrow \exp(-\infty) = 0$$

So every point except the closest one get weight ≈ 0

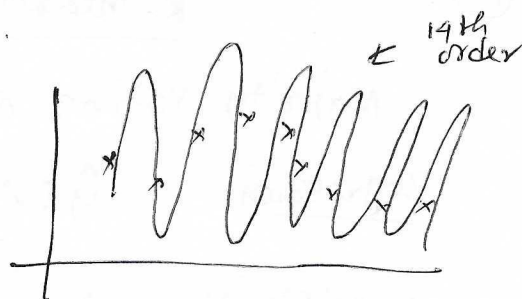
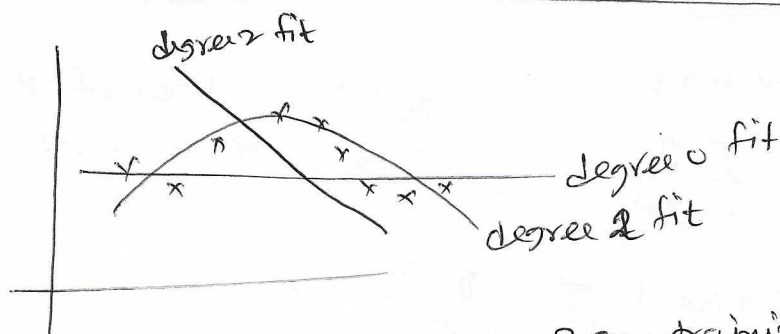
If $x = x_i$, $w_i = 1$

Thus, $\lim_{\sigma \rightarrow 0} w_i = \begin{cases} 1 & \text{if } x = x_i \\ 0 & \text{if } x \neq x_i \end{cases}$

so its creating a delta function (spike) at the 1-NN.

The decay function works in between the two extremes.

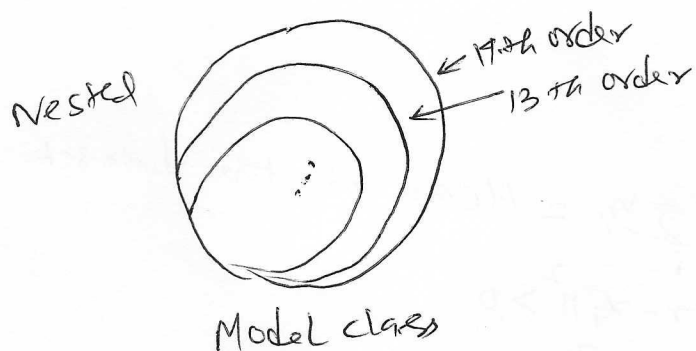
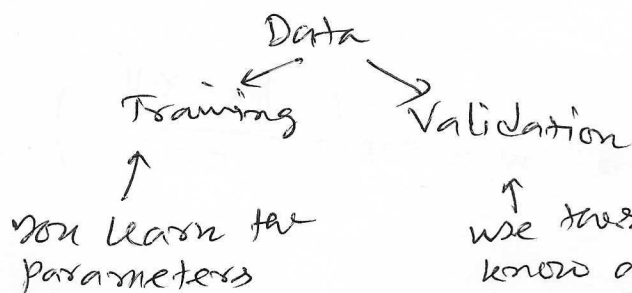
Bias, Variance, Cross-validation.



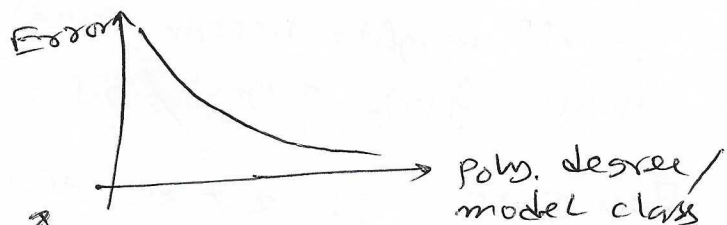
zero training error !!

on which ground would you reject that??

It's not that if you do well in train data; it matters how do you perform on test data.



If you use larger model class, then your training error will go down



But this is not an indication on how well you will perform on new data.

Expected Loss vs. Empirical Loss

$$X, Y \sim P(X, Y)$$

we have about expected loss $\rightarrow$ average loss over the true data distribution. Average error your model would make over infinitely many samples drawn from real world distribution.

$$E_{P(X, Y)} [L(y, \hat{y})] \quad \leftarrow \text{we want to minimize}$$

$$\min_{\vec{w}} E_{P(X, Y)} [L(y, \hat{y})]$$

$$= \min_{\vec{w}} \int \int L(y, \hat{y}) P(x, y) dy$$

$\leftarrow$ very hard to compute !!

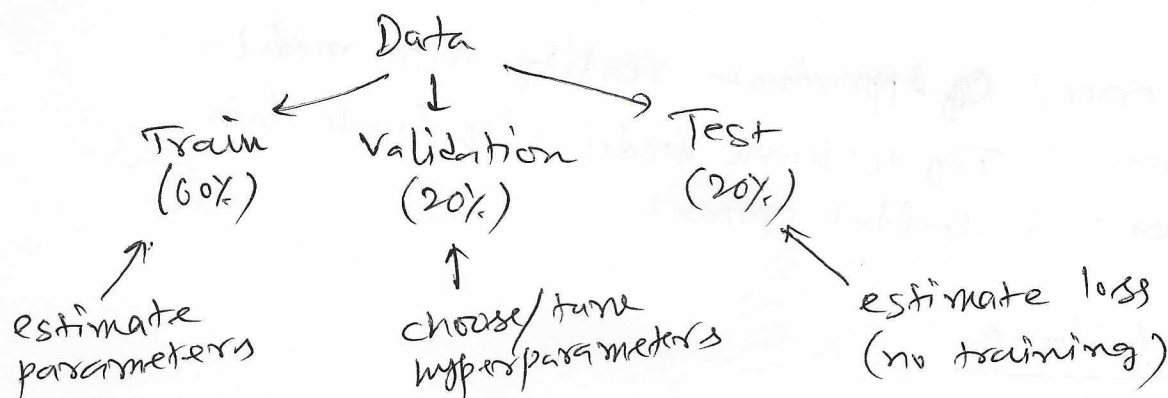
So we replace the double integral with samples:

$$(x_i, y_i) \sim p(x, y)$$

$$E_{\text{approx}} = \frac{1}{N} \sum_{i=1}^N L(y_i, \hat{y}_i) \leftarrow \text{Empirical loss}$$

(average error over training samples)

We try to minimize empirical loss with the hope that it will approximate expected loss.



Training loss $\rightarrow$ Empirical loss

Validation/Test loss $\rightarrow$ estimate of Expected loss.

Cross validation

Problem with small data size. Break your data into chunks. then consider one part as train, other part as validation.

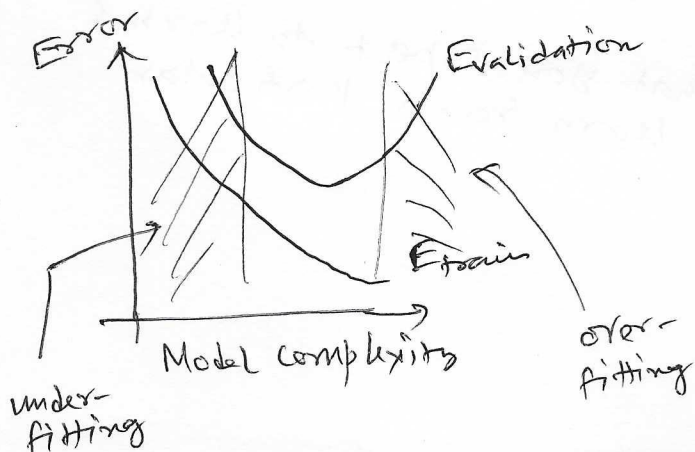


K-fold cross-validation

Partition the data into k roughly equal parts
 Train on all but j -th part, test on j -th part.

& repeat on different chunks.

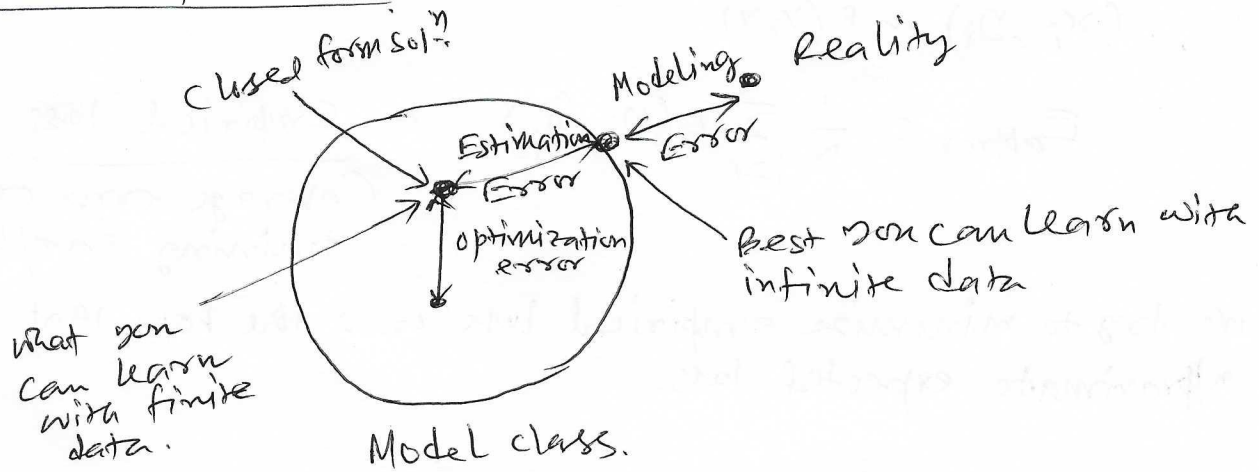
Leave-one-out Cross Validation: use one data, & use rest for other chunk.



Underfitting: too simple model

Overfitting: too complex model

Error decomposition

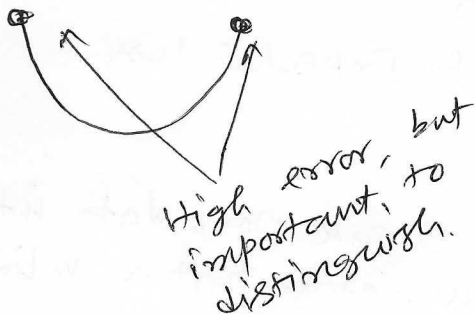


Modeling error: Approximate reality with model

Estimation " : Try to learn model with finite data

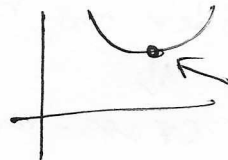
Optimization " : Couldn't optimize.

Bias & Variance



$$\text{Error} \Rightarrow \text{Bias (modeling error)} + \text{Variance (Estimation error)}$$

Bias-Variance trade-off

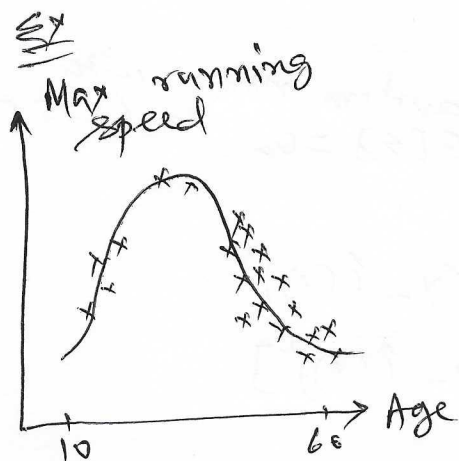


Poly. degree 1 $\rightarrow$ fewer parameters $\rightarrow$ need less data to estimate $\rightarrow$ low variance, high bias

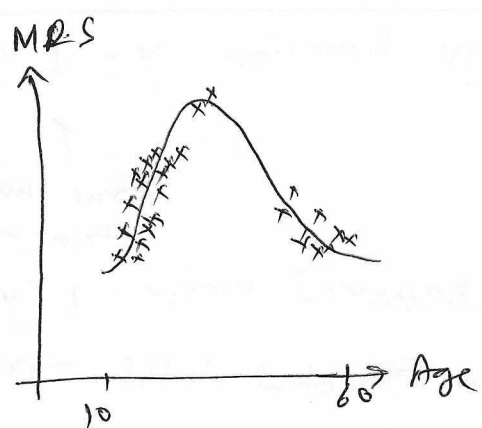
Poly. degree 100 $\rightarrow$ more parameters $\rightarrow$ low bias, high variance.

Bias $\rightarrow$ what you expect to learn from ground truth. (empirical loss)

Variance $\rightarrow$ Difference between what you expect to learn & what you actually learn from a particular dataset.

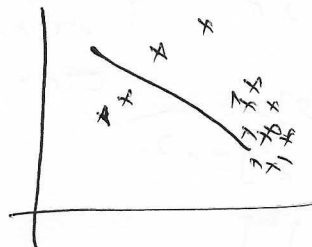


Dataset #1

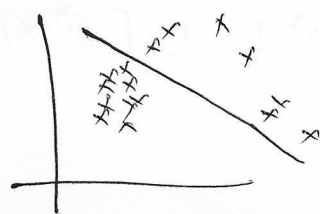


Dataset #2

Linear regression: (low variance, high bias)



Dataset #1



Dataset #2

(bad model)

15th order poly. fit:

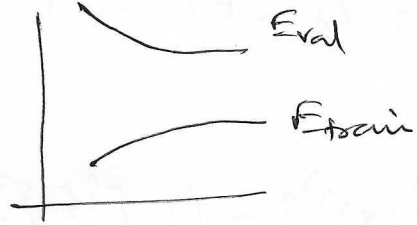
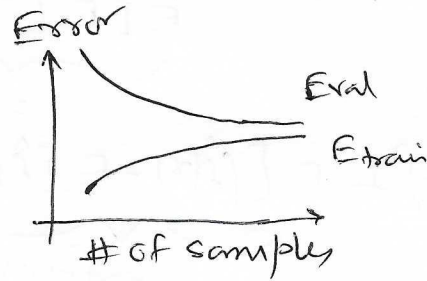


(behaves very differently in 2 datasets)

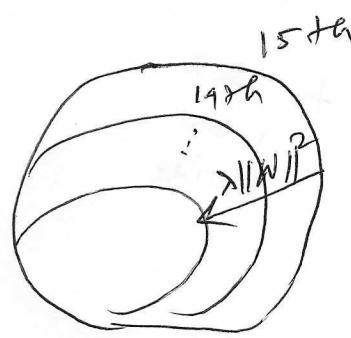
(high variance, low bias)

More complex class $\rightarrow$ less bias, more variance

under-fitting



over-fitting



Effect of regularization on model classes.

Error decomposition: derivation

True function $y = f^*(x) + \varepsilon$ $\leftarrow$ random noise with $E[\varepsilon] = 0, \text{Var}(\varepsilon) = \sigma^2$
 $\uparrow$
true underlying function

Loss: squared error: $L(y, \hat{f}(x)) = (y - \hat{f}(x))^2$

We want expected total error: $E[(y - \hat{f}(x))^2]$

Now, $y = f^*(x) + \varepsilon$

$$\text{So, } E[(y - \hat{f}(x))^2] = E[(f^*(x) + \varepsilon - \hat{f}(x))^2]$$

$$\text{Since } (f^*(x) - \hat{f}(x) + \varepsilon)^2 = (f^*(x) - \hat{f}(x))^2 + 2(f^*(x) - \hat{f}(x))\varepsilon + \varepsilon^2,$$

$$\text{We have, } E[(y - \hat{f}(x))^2] = E[(f^*(x) - \hat{f}(x))^2] + \sigma^2 \quad \left(\begin{array}{l} \text{taking the} \\ \text{expectation} \\ \text{over } \varepsilon \end{array} \right)$$

$E[\varepsilon] = 0$
 $E[\varepsilon^2] = \sigma^2$

Now focus on the first term: $E[(f^*(x) - \hat{f}(x))^2]$

Add & subtract $E[\hat{f}(x)]$:

$$f^*(x) - \hat{f}(x) = \underbrace{f^*(x) - E[\hat{f}(x)]}_{\text{bias}} + \underbrace{E[\hat{f}(x)] - \hat{f}(x)}_{\text{variance}}$$

$$\Rightarrow (f^* - \hat{f})^2 = (f^* - E[\hat{f}])^2 + 2(f^* - E[\hat{f}])(E[\hat{f}] - \hat{f}) + (\hat{f} - E[\hat{f}])^2$$

Now take expectation over the training set:

$$E[(f^* - \hat{f})^2] = (f^* - E[\hat{f}])^2 + E[(\hat{f} - E[\hat{f}])^2] + 2(f^* - E[\hat{f}]) \underbrace{E[E[\hat{f}] - \hat{f}]}_{=0}$$

$$\text{So, } E[(y - \hat{f}(x))^2] = \underbrace{(E[\hat{f}(x)] - f^*(x))^2}_{\text{bias}^2} + \underbrace{E[(\hat{f}(x) - E[\hat{f}(x)])^2]}_{\text{variance}}$$

+ σ^2
Noise

$$\text{So, } \boxed{\text{Error} = \text{bias}^2 + \text{variance} + \text{noise}}$$